

Práctica 3: Cepstrum y análisis homomórfico

1)

Sintetice la vocal /ae/ como se explica en el ejercicio 3 de la guía 3. Debe mostrarse un gráfico de la señal temporal y de su espectro. Verifique que los picos de la envolvente coinciden con los datos usados para generar la señal.

Datos iniciales(ej3-guia)

- Frecuencia fundamental:

$F_0 = 125, \text{Hz}$

- Periodo de muestreo:

$T = \frac{1}{16000}, \text{s}$

- Formantes (frecuencias de resonancia y ancho de banda aproximado σ):

$F_k, \sigma_k = (660, 60), (1720, 100), (2410, 120), (3500, 175), (4500, 250)$

- Modelo de radiación:

$R(z) = 1 - \gamma z^{-1}, \quad \gamma = 0.96$

- Excitación: tren de impulsos (sin modelo glotal).

Excitación

Un tren de impulsos de periodo N_0 (en muestras), con:

$$N_0 = \frac{F_s}{F_0} = \frac{16000}{125} = 128$$

Es decir, cada 128 muestras hay un 1 (impulso), el resto ceros:

$$p(n) = \sum_{m=-\infty}^{\infty} \delta(n - mN_0)$$

Nota: Tiene que estar multiplicado por un factor de decaimiento exponencial debido a que una señal infinita puede traer problemas de convergencia en el cepstrum.

$$p(n) = \sum_{m=-\infty}^{\infty} \beta^m \delta(n - mN_0)$$

En frecuencia, su espectro es una serie de líneas espaciadas cada F_0 .

Tracto vocal

La función del tracto vocal se modela como un producto de filtros resonantes de segundo orden:

$$V(z) = \prod_{k=1}^5 \frac{1}{1 - 2e^{-2\pi\sigma_k T} \cos(2\pi F_k T) z^{-1} + e^{-4\pi\sigma_k T} z^{-2}}$$

Cada término genera un pico en F_k con ancho relacionado a σ_k .

Radiación

El modelo de radiación labial es:

$$R(z) = 1 - 0.96z^{-1}$$

Que corresponde a un cero cercano a $z = 1$, modelando la diferenciación.

```
In [1]: import numpy as np
import numpy.fft as ft
import matplotlib.pyplot as plt

import scipy.io as io
import scipy.io.wavfile as wav
import scipy.signal as sig
import scipy.linalg as la

import IPython.display as ipd
import seaborn as sns
import sympy as sym

from IPython.display import Audio
import seaborn as sns

from google.colab import files
import soundfile as sf

from mpl_toolkits.axes_grid1.inset_locator import inset_axes, mark_inset
```

```
In [2]: #Parametros.
Fk = np.array([660, 1720, 2410, 3500, 4500])
ok = np.array([60, 100, 120, 175, 250])
Fs = 16_000
F0 = 125 #hz
T = 1/Fs #s
gamma = 0.96

Nfft = 8192 * 4 # para espectro
Dur = 1.0 # 1 segundo de señal
N = int(Dur*Fs)
N0 = int(Fs/F0) # 128 muestras
#print(N0)
beta=0.999
```

```
In [3]: #Señal de entrada
p = np.zeros(N)
p[:N0] = 1.0
```

```

p = p * beta**(np.arange(N)/N0)

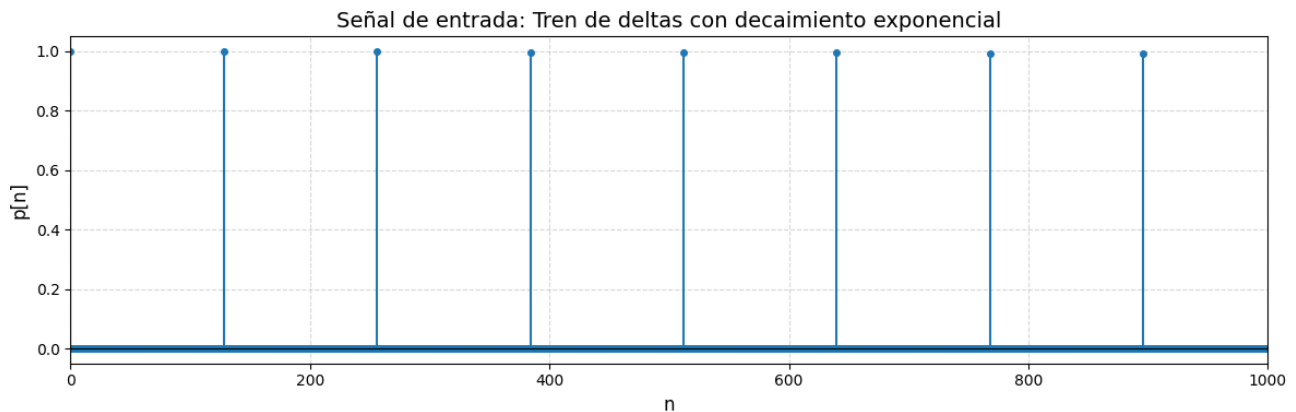
fig, ax = plt.subplots(figsize=(12, 4))
markerline, stemlines, baseline = ax.stem(np.arange(N), p, linefmt='C0-', markerfmt='C0o', basefmt='k-')

plt.setp(markerline, markersize=4)
plt.setp(stemlines, linewidth=1.5)
plt.setp(baseline, linewidth=0.8)

ax.set_xlim(0, 1000)
ax.set_xlabel('n', fontsize=12)
ax.set_ylabel('p[n]', fontsize=12)
ax.set_title("Señal de entrada: Tren de deltas con decaimiento exponencial", fontsize=14)
ax.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()

```



Tracto vocal (cascada de resonadores)

Cada formante es un filtro de segundo orden:

$$V_k(z) = \frac{1}{1 - 2\rho_k \cos(\theta_k)z^{-1} + \rho_k^2 z^{-2}}$$

donde:

$$\rho_k = e^{-2\pi\sigma_k T}, \quad \theta_k = 2\pi F_k T$$

```

In [4]: class ceptralVocalTract:
        """implementa V(w) y R(w) luego los combina para H(w)"""
        def __init__(self, fk, sigmak, gamma, T):
            self.fk = fk
            self.sigmak = sigmak
            self.T = T
            self.y = gamma

        def vocalTract(self, source):
            """V(w)"""
            v = source.copy()
            for f_, o in zip(self.fk, self.sigmak):
                p = np.e**(-2*np.pi*o*T)
                theta = 2*np.pi*f_*T
                num = [1.0]
                den = [1.0, -2*p*np.cos(theta), p*p]
                v = sig.lfilter(num, den, v)
            return v

        def labialRadiation(self, source):
            """R(w)"""
            return sig.lfilter([1, -self.y], [1], source)

        def transform(self, source):
            """H(w)"""
            return self.labialRadiation(self.vocalTract(source))

```

La síntesis se da a través de la convolución de $p[n] * r[n] * v[n]$ o en el espectro

$$S[z] = P[z] R[z] V[z]$$

```

In [5]: # delta(n)
delta = np.zeros(N); delta[0]=1.0
tracto_H = ceptralVocalTract(Fk, ok, gamma, T)
#f-para tener en frecuencia a los ejes.
f = np.linspace(0, Fs, Nfft, endpoint=False)

h = tracto_H.transform(delta)
s = tracto_H.transform(p)

#V[z], H[z], S[z]
#V = ft.fft(v, Nfft) #1.V
H = ft.fft(h, Nfft) #1.V.R
S = ft.fft(s, Nfft) #P.V.R

```

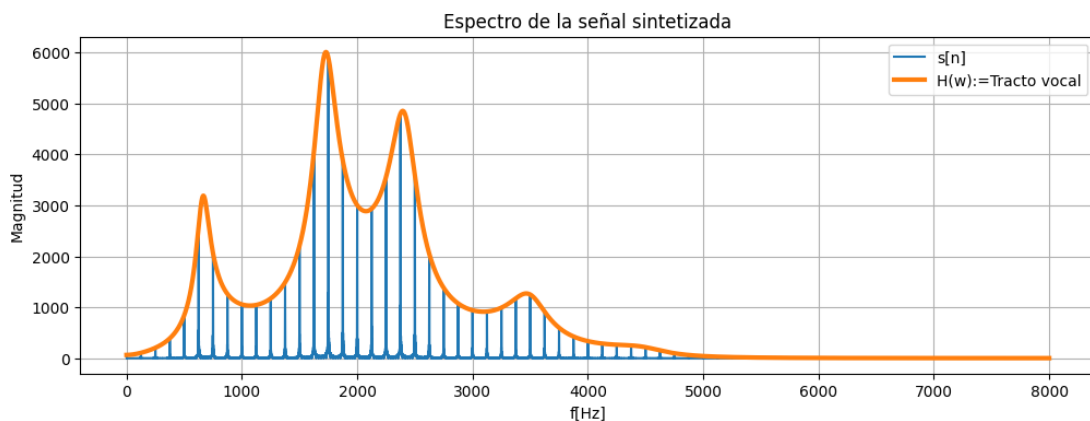
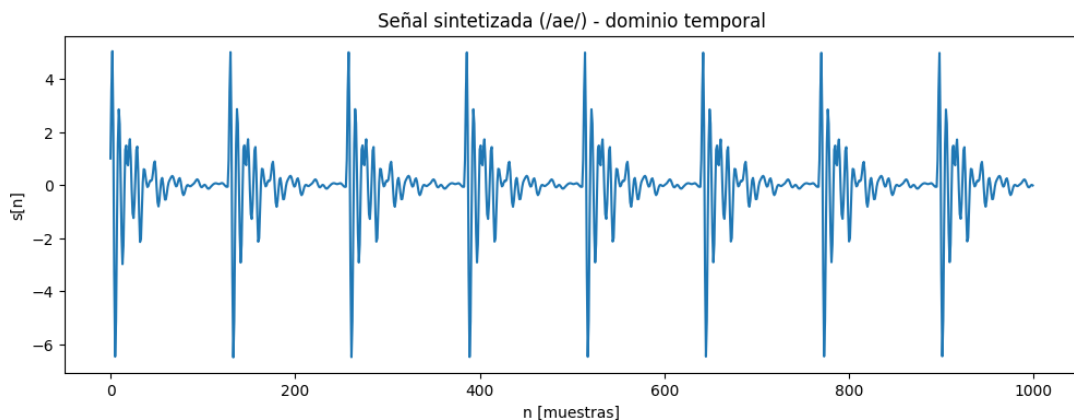
```

In [6]: # Señal temporal
plt.figure(figsize=(12,4))
plt.plot(s[:1000])
plt.title("Señal sintetizada (/ae/) - dominio temporal")
plt.xlabel("n [muestras]")
plt.ylabel("s[n]")
plt.show()

plt.figure(figsize=(12,4))
plt.plot(f[:Nfft//2], np.abs(S[:Nfft//2]), label='s[n]')
plt.plot(f[:Nfft//2], sum(p)*np.abs(H[:Nfft//2]), lw=3, label='H(w):=Tracto vocal') #deberia estar multiplicado por algo que deje la misma energia??
plt.title("Espectro de la señal sintetizada")
plt.xlabel("f[Hz]")
plt.ylabel("Magnitud")

```

```
plt.legend()
plt.grid()
plt.show()
```



In [7]: `Audio(s, rate=Fs)`

Out[7]: 0:00 / 0:01

2)

Calcule el cepstrum de la señal con la función de cepstrum construida a partir de lo explicado en las prácticas. En base a este cálculo grafique solamente la envolvente de la señal sintetizada. Muestre en el mismo gráfico la envolvente verdadera de la señal correspondiente a $V(\omega)R(\omega)$.

```
In [8]: Nceps = 8192*16
s_cepstrum = 2*ft.ifft(np.log(np.abs(ft.fft(s, Nceps))))
p_cepstrum = 2*ft.ifft(np.log(np.abs(ft.fft(p, Nceps))))
h_cepstrum = 2*ft.ifft(np.log(np.abs(ft.fft(h, Nceps))))

#Se define un eje de frecuencia para cepstrum, con mas puntos
f_cepstrum = np.linspace(0, Fs, Nceps, endpoint=False)
```

Sabemos que $H(w) = V(w) R(w)$ que sería la envolvente y que $s[n] = p[n] * h[n]$, ahora con cepstrum se tiene

$$\begin{aligned}\hat{s}[n] &= \hat{p}[n] + \hat{h}[n] \\ &= \hat{p}[n] + \hat{o}[n] + \hat{r}[n]\end{aligned}\quad \begin{matrix} (1) \\ (2) \end{matrix}$$

In [9]: `s_hat = p_cepstrum.real + h_cepstrum.real`

```
In [10]: sns.set(style="whitegrid", context="talk", palette="deep")
fig, ax = plt.subplots(2, 1, figsize=(15, 6), sharex=True)

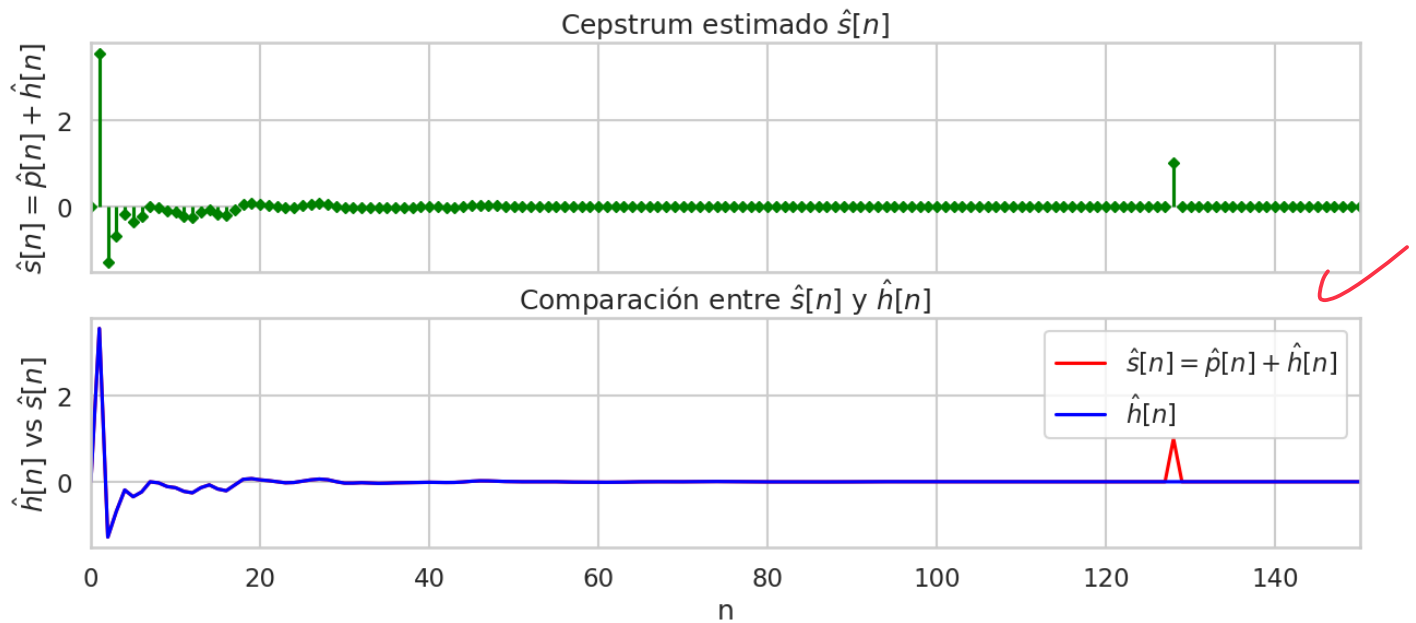
fig.suptitle('Vistazo cepstrum temporal-no obligatorio', fontsize=16, y=1.02)
markerline, stemlines, baseline = ax[0].stem(s_hat, linefmt='green', markerfmt='D', basefmt=" ")
plt.setp(markerline, markersize=5)
ax[0].set_xlim(0, 150)
#ax[0].set_xlabel('n')
ax[0].set_ylabel(r'$\hat{s}[n] = \hat{p}[n] + \hat{h}[n]$')
ax[0].set_title(r'Cepstrum estimado $\hat{s}[n]$')
#
ax[1].plot(s_hat, color='red', label=r'$\hat{s}[n] = \hat{p}[n] + \hat{h}[n]$')
ax[1].plot(h_cepstrum.real, color='blue', label=r'$\hat{h}[n]$')
ax[1].set_xlim(0, 150)
ax[1].set_xlabel('n')
ax[1].set_ylabel(r'$\hat{h}[n]$ vs $\hat{s}[n]$')
ax[1].set_title(r'Comparación entre $\hat{s}[n]$ y $\hat{h}[n]$')
ax[1].legend()
```

Out[10]: <matplotlib.legend.Legend at 0x7c272bf44ef0>

idm

la idea es compararla con s-cepstrum

Vistazo cepstrum temporal-no obligatorio



```
In [33]: """
fig, ax = plt.subplots(1, 1, figsize=(10, 6), sharex=True)

#S_hat(w). H_cep(w)
#ax.plot(f_cepstrum, np.abs(np.exp(ft.fft(s_hat))), color='red', label=r'\hat{S}[w] = e^{fft(\hat{p}[n] + \hat{h}[n])}')
ax.plot(f, np.sum(p)*np.abs(H), color='red', label=r'$H[w] = fft(h[n])$')
ax.plot(f_cepstrum, np.abs(np.exp(ft.fft(h_cepstrum, Nceps))), color='blue', label=r'\hat{H}[w] = e^{\hat{h}[n]}$')
ax.set_xlim(0, 3000)
ax.set_xlabel('w')
ax.set_ylabel(r'\hat{H}(w) vs \hat{S}(w)')
ax.set_title(r'Comparación entre \hat{S}(w) y \hat{H}(w)')
ax.legend()
"""
print(" ")
```

```
<>:5: SyntaxWarning: invalid escape sequence '\h'
<>:5: SyntaxWarning: invalid escape sequence '\h'
/tmp/ipython-input-1577849182.py:5: SyntaxWarning: invalid escape sequence '\h'
#ax.plot(f_cepstrum, np.abs(np.exp(ft.fft(s_hat))), color='red', label=r'\hat{S}[w] = e^{fft(\hat{p}[n] + \hat{h}[n])}')

Se puede notar que en  $n = 80$  ya la parte de  $h[n]$  se extingue, se puede cortar ahí para hacer el liftering
```

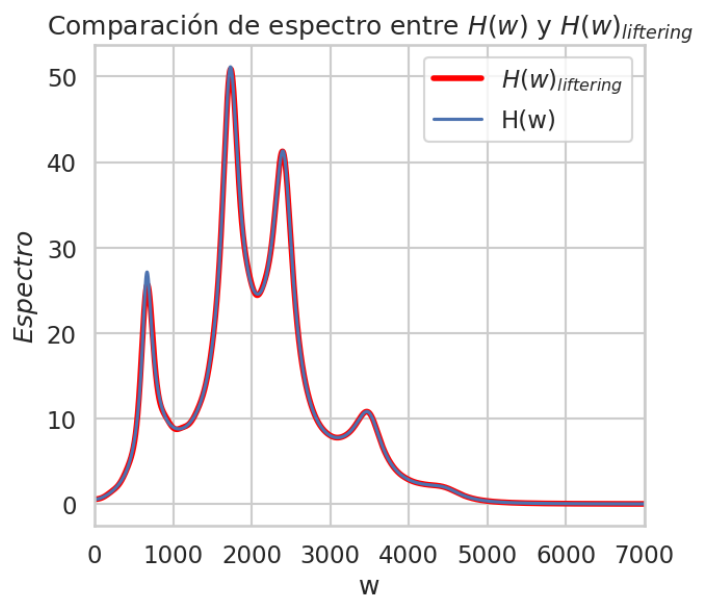
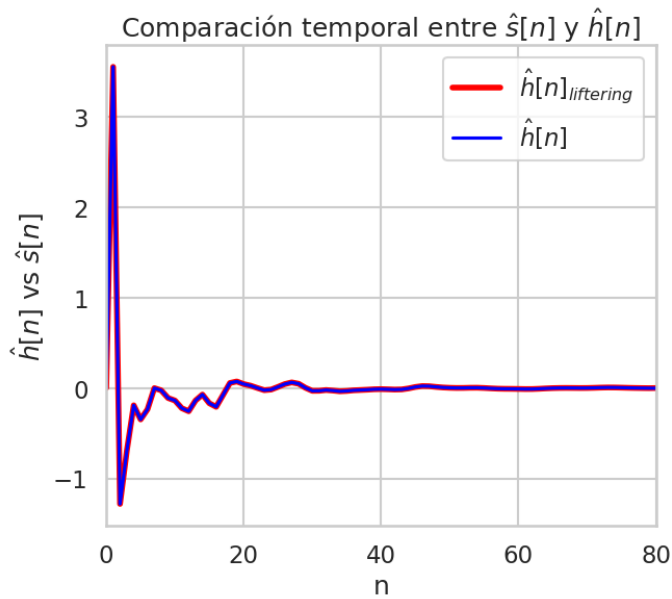
```
In [12]: n_lift = 80

h_liftering = np.zeros(s_hat.shape)
h_liftering[:n_lift] = s_hat[:n_lift] ~ debes usar s_cepstrum

H_lifterin = np.exp(ft.fft(h_liftering, Nceps))

fig, ax = plt.subplots(1, 2, figsize=(15, 6), sharex=False)
ax[0].plot(h_liftering.real, color='red', lw=4, label=r'\hat{h}[n]_{liftering}$')
ax[0].plot(h_cepstrum.real, color='blue', label=r'\hat{h}[n]$')
ax[0].set_xlim(0, 80)
ax[0].set_xlabel('n')
ax[0].set_ylabel(r'\hat{h}[n] vs \hat{s}[n]')
ax[0].set_title(r'Comparación temporal entre \hat{s}[n] y \hat{h}[n]')
ax[0].legend()
#
ax[1].plot(f_cepstrum, np.abs(H_lifterin), color='red', lw=4, label=r'$H(w)_{liftering}$')
ax[1].plot(f, np.abs(H), label='H(w)')
ax[1].set_xlim(0, 7000)
ax[1].set_xlabel('w')
ax[1].set_ylabel(r'$Especros$')
ax[1].set_title(r'Comparación de espectro entre $H(w)$ y $H(w)_{liftering}$')
ax[1].legend()
```

```
Out[12]: <matplotlib.legend.Legend at 0x7c271bcc2b40>
```



Notar que ya con $N = 80$ ya se puede apreciar un *liftering* muy bueno, es decir puedo separar $\hat{h}[n]$ de $\hat{p}[n]$ y obtener $h[n]$.

3)

Sintetice una vocal diferente a la anterior correspondiente al espectro de una señal emitida real. Para ello, grabe la señal con su propia voz, seleccione alguna porción correspondiente a una vocal, genere su espectro y luego adapte los parámetros de la vocal/ae/ para que representen el espectro de esta vocal. Utilice dichos parámetros para generar una vocal sintética. Compare el espectro real con el sintético.

```
In [13]: !wget -O fomema_o.wav https://raw.githubusercontent.com/Alex-f98/procHabla20252c/main/ooooo.wav #autocontenido jeje

Out[13]: ['-2025-10-03 22:42:01-- https://raw.githubusercontent.com/Alex-f98/procHabla20252c/main/ooooo.wav',
'Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...',
'Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.',
'HTTP request sent, awaiting response... 200 OK',
'Length: 413740 (404K) [audio/wav]',
'Saving to: 'fomema_o.wav'',
'',
'fomema_o.wav      0%[ ] 0 --KB/s ',
'fomema_o.wav     100%[=====] 404.04K --KB/s in 0.03s ',
'',
'2025-10-03 22:42:01 (15.4 MB/s) - 'fomema_o.wav' saved [413740/413740]',
'']
```

```
In [14]: x, Fs = sf.read("fomema_o.wav")
x = x / np.max(np.abs(x))
x_seg = x[160_000:200_000]

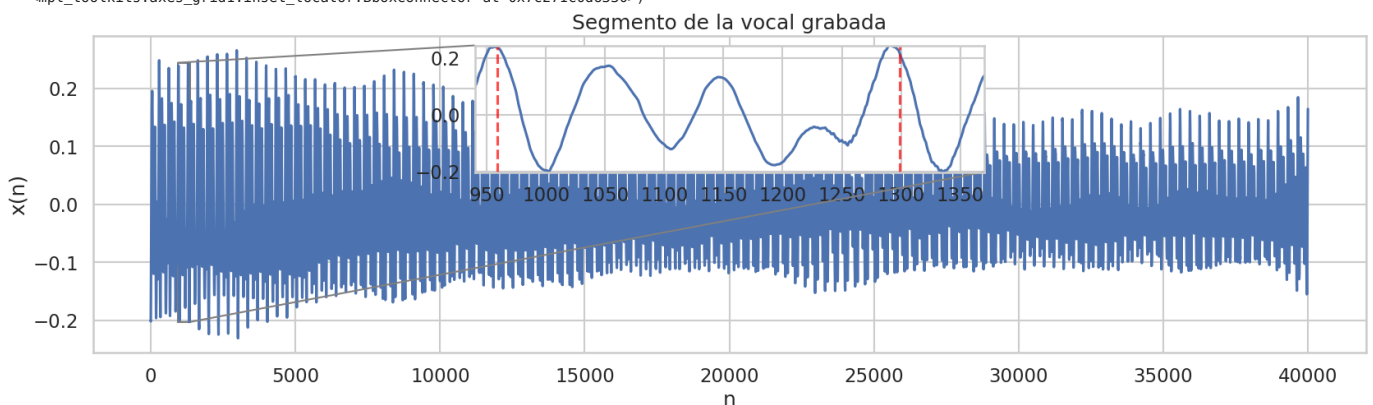
fig, ax = plt.subplots(figsize=(20, 5))
ax.plot(x_seg)
ax.set_title("Segmento de la vocal grabada")
ax.set_ylabel("x(n)")
ax.set_xlabel("n")

# zoom-inset
axins = inset_axes(ax, width="40%", height="40%", loc="upper center") # tamaño y ubicación del zoom
axins.plot(x_seg)
for freq in [970-11, 1310-11]:
    axins.axvline(x=freq, color='red', linestyle='--', alpha=0.7)

x1, x2 = 940, 1370
axins.set_xlim(x1, x2)
axins.set_ylim(min(x_seg[x1:x2]), max(x_seg[x1:x2]))
mark_inset(ax, axins, loc1=2, loc2=4, fc="none", ec="0.5")
```

✓ qué lindo !!

```
Out[14]: (<mpl_toolkits.axes_grid1.inset_locator.BboxPatch at 0x7c271bd577d0>,
<mpl_toolkits.axes_grid1.inset_locator.BboxConnector at 0x7c271c14f9e0>,
<mpl_toolkits.axes_grid1.inset_locator.BboxConnector at 0x7c271c0d6330>)
```



Para generar el tren de pulsos sintético tengo que encontrar F_0 la frecuencia en la que vibran las cuerdas vocales o lo que es lo mismo T_0 el pitch.

```
In [15]: mi_F0 = 1/((1310 - 970)/Fs)
print(f"mi F0 es aproximadamente: {mi_F0}")

mi_F0 es aproximadamente: 129.7058823529412
```

Luego hay que ver donde están los parámetros formantes, defino por F_k y sus anchos relacionados con σ_k dados en el primer ejercicio.

Estos deberían ser similares a los obtenidos (del mismo orden) y en este caso uso tres formantes (por el ejemplo o.txt)

Para ello también hay que redefinir Nfft dado que ahora la Fs es mayor.

```
In [16]: #Redefino Nfft dado que hay
Nfft = 4*88192
f = np.linspace(0, Fs, Nfft, endpoint=False)

SX = np.abs(ft.fft(x_seg, Nfft))

plt.figure(figsize=(10,4))
plt.plot(f, SX)

#1er ejercicio
#f_x = [200, 480, 2100]
#o_x = [60, 100, 120]

f_x = [100, 480, 2100]
o_x = [140, 50, 100]

for freq in f_x:
    plt.axvline(x=freq, color='red', linestyle='--', alpha=0.7, label=f'Formante en w={freq:.3f}')

plt.xlim((0,3000))
plt.title("Espectro de la vocal [0] real")
plt.xlabel("f")
plt.ylabel("Magnitud")
plt.grid(True, which='both')
plt.show()
```



```
In [17]: #Descomentar para calcular F0 con librosa, no es muy distinto al calculado a mano pero me quedo con este.
"""
!!pip install librosa

import librosa

f0, voiced_flag, voiced_prob = librosa.pyin(
    x_seg.astype(float),
    fmin=50,
    fmax=500,
    sr=Fs
)

f0_valid = f0[~np.isnan(f0)]
F0_librosa = np.median(f0_valid)
print(f"F0 estimado (librosa.pyin): {F0_librosa:.2f} Hz")
"""
```

```
Out[17]: 'n!!pip install librosa\n\nimport librosa\n\nf0, voiced_flag, voiced_prob = librosa.pyin(\n    x_seg.astype(float),\n    fmin=50,\n    fmax=500,\n    sr=Fs\n)\n\nf0_valid = f0[~np.isnan(f0)]\nF0_librosa = np.median(f0_valid)\nprint(f"F0 estimado (librosa.pyin): {F0_librosa:.2f} Hz")\n'
```

```
In [18]: F0 = 130.44 # librosa; 129.706 estimado a ojo
T = 1/Fs # Pitch
N0 = int(Fs/F0)
N = len(x_seg)
beta = 0.999
gamma = 0.98

#Tren de deltas con decaimiento exponencial.
p = np.zeros(N); p[::N0] = 1.0
p = p * beta**(np.arange(N)/N0)

#redefino delta por nuevo N.
delta = np.zeros(N); delta[0] = 1.0

#Nfft = 4*88192
#f = np.linspace(0, Fs, Nfft)
```

Se ajustaron los σ_k lo menor posible a fin de ajustar la envolvente al espectro real y se obtiene lo siguiente.

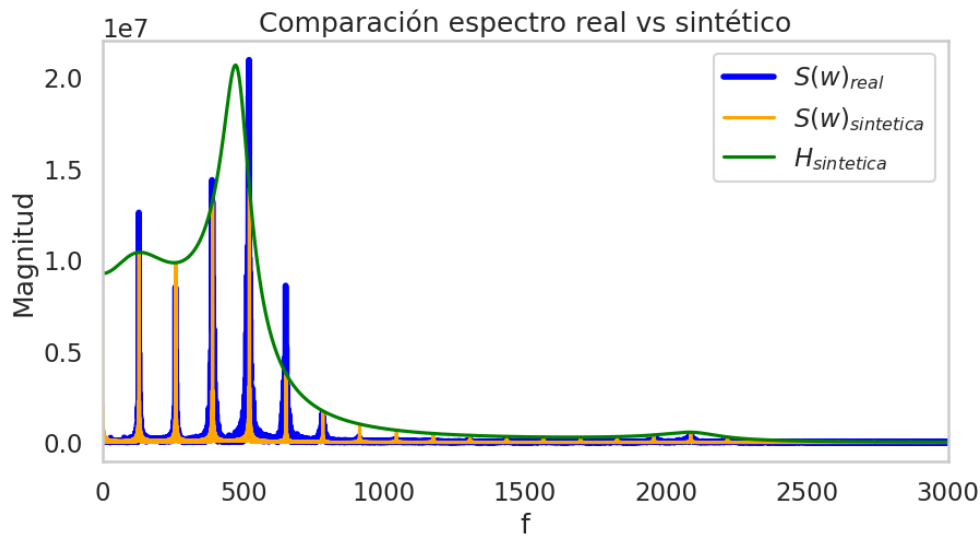
```
In [19]: tracto_H_brian = cepstralVocalTract(f_x, o_x, gamma, T)
h_syn = tracto_H_brian.transform(delta)
s_syn = tracto_H_brian.transform(p)

S_syn = np.fft.fft(s_syn, Nfft)
H_syn = np.fft.fft(h_syn, Nfft)

plt.figure(figsize=(10,5))
plt.plot(f, 2e4*np.abs(SX), lw=4, c="blue", label=r"$S(w)_{\text{real}}$")
plt.plot(f, np.abs(S_syn), c="orange", label=r"$S(w)_{\text{sintetica}}$")
plt.plot(f, np.sum(p)*np.abs(H_syn), c="green", label=r"$H_{\text{sintetica}}$")
#plt.xlim((0,np.pi/2))
plt.xlim((0, 3000))

plt.xlabel("f")
plt.ylabel("Magnitud")
plt.title("Comparación espectro real vs sintético")
plt.legend()
```

```
plt.grid()
plt.show()
```



!! muy bueno !!

Señal sintetizada

```
In [20]: Audio(s_syn, rate=Fs)
```

Out[20]: 0:00 / 0:01

Señal real (4s)

```
In [21]: Audio(x, rate=Fs)
```

Out[21]: 0:00 / 0:05

```
In [22]: sf.write('brian_syn_o.wav', s_syn, Fs)
```

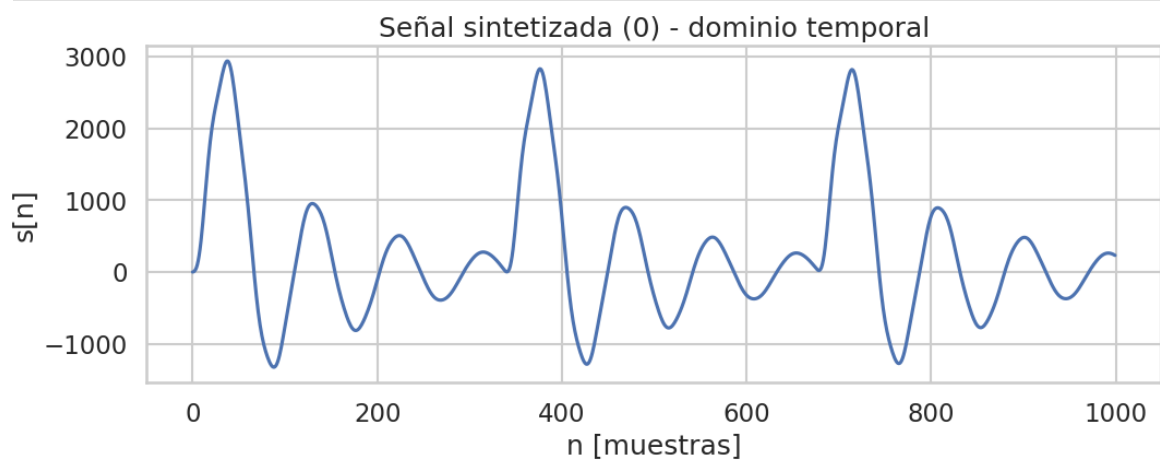
4)

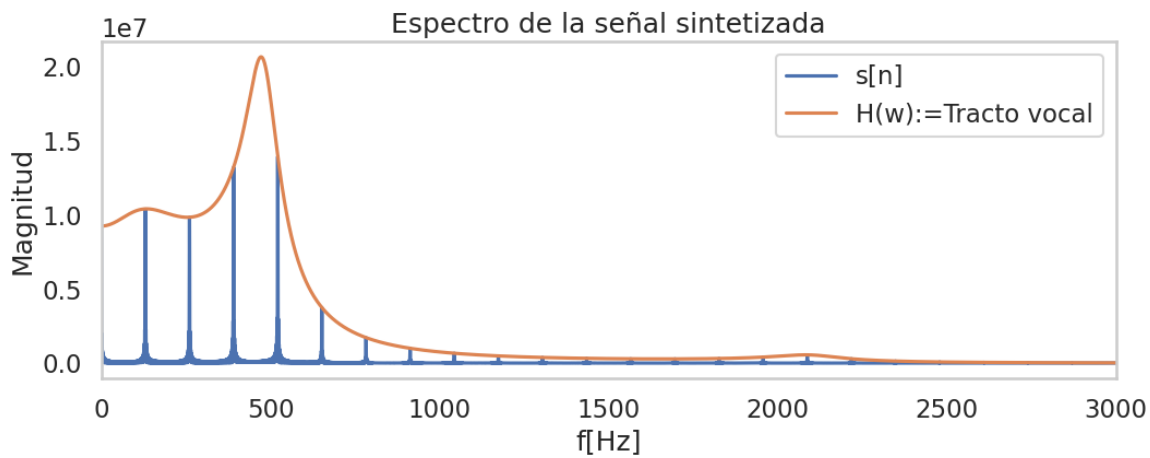
Repetir los 2 primeros ítems sobre esta última vocal sintética y sobre la grabada.

1)

```
In [23]: # Señal temporal
plt.figure(figsize=(12,4))
plt.plot(s_syn[:1000])
plt.title("Señal sintetizada (0) - dominio temporal")
plt.xlabel("n [muestras]")
plt.ylabel("s[n]")
plt.show()

plt.figure(figsize=(12,4))
plt.plot(f[:Nfft//2], np.abs(S_syn[:Nfft//2]), label='s[n]')
plt.plot(f[:Nfft//2], sum(p)*np.abs(H_syn[:Nfft//2]), label='H(w):=Tracto vocal') #deberia estar multiplicado por algo que deje la misma energia??
plt.xlim((0,3000))
plt.title("Espectro de la señal sintetizada")
plt.xlabel("f[Hz]")
plt.ylabel("Magnitud")
plt.legend()
plt.grid()
plt.show()
```





2)

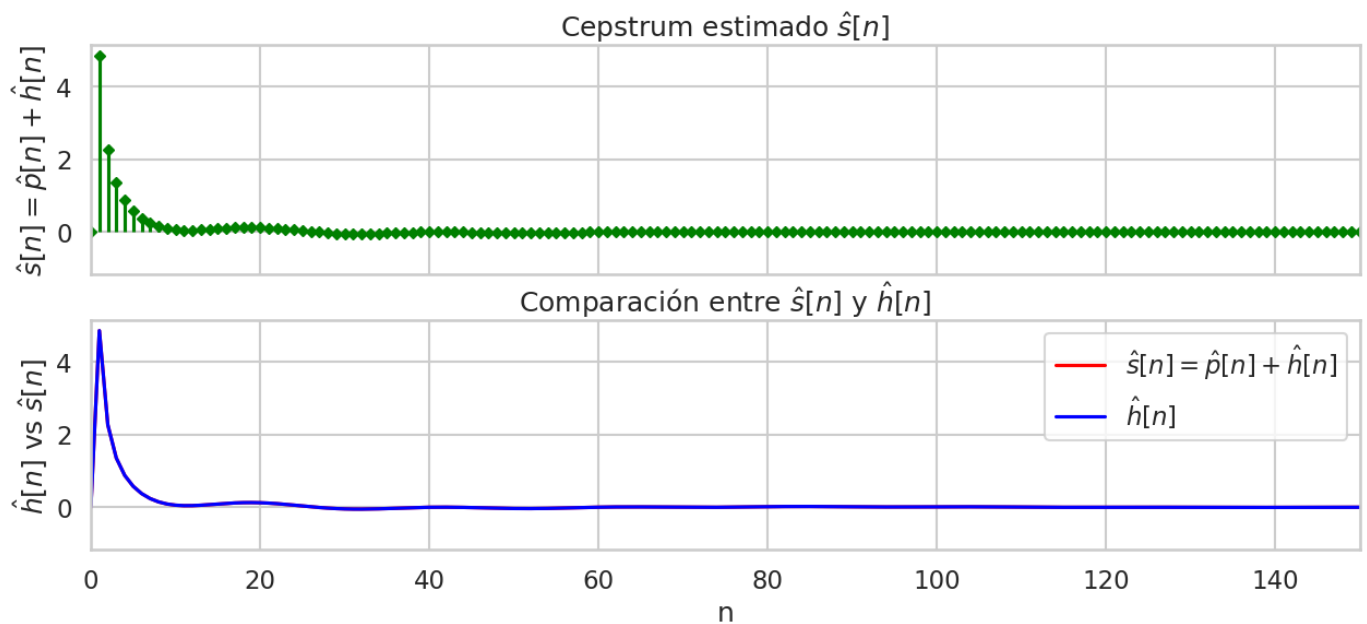
```
In [24]: Nceps = 88192*16
s_cepstrum = 2*fft.ifft(np.log(np.abs(fft.fft(s_syn, Nceps))))
p_cepstrum = 2*fft.ifft(np.log(np.abs(fft.fft(p_syn, Nceps))))
h_cepstrum = 2*fft.ifft(np.log(np.abs(fft.fft(h_syn, Nceps))))

s_hat = p_cepstrum.real + h_cepstrum.real
sns.set(style="whitegrid", context="talk", palette="deep")

fig, ax = plt.subplots(2, 1, figsize=(15, 6), sharex=True)

markerline, stemlines, baseline = ax[0].stem(s_hat, linefmt='green', markerfmt='D', basefmt=" ")
plt.setp(markerline, markersize=5)
ax[0].set_xlim(0, 150)
ax[0].set_xlabel('n')
ax[0].set_ylabel(r'$\hat{s}[n] = \hat{p}[n] + \hat{h}[n]$')
ax[0].set_title(r'Cepstrum estimado $\hat{s}[n]$')
#
ax[1].plot(s_hat, color='red', label=r'$\hat{s}[n] = \hat{p}[n] + \hat{h}[n]$')
ax[1].plot(h_cepstrum.real, color='blue', label=r'$\hat{h}[n]$')
ax[1].set_xlim(0, 150)
ax[1].set_xlabel('n')
ax[1].set_ylabel(r'$\hat{h}[n]$ vs $\hat{s}[n]$')
ax[1].set_title(r'Comparación entre $\hat{s}[n]$ y $\hat{h}[n]$')
ax[1].legend()
```

Out[24]: <matplotlib.legend.Legend at 0x7c26ae89f710>



Se elige un $n = 120$, mayor al de los primeros ejercicios para lograr un mejor resultado.

```
In [25]: n_lift = 120
h_liftering = np.zeros(s_hat.shape)
h_liftering[:n_lift] = s_hat[:n_lift]
f_cepstrum = np.linspace(0, Fs, Nceps)

H_lifterin = np.exp(fft.fft(h_liftering, Nceps))

In [26]: fig, ax = plt.subplots(1, 2, figsize=(15, 6), sharex=False)

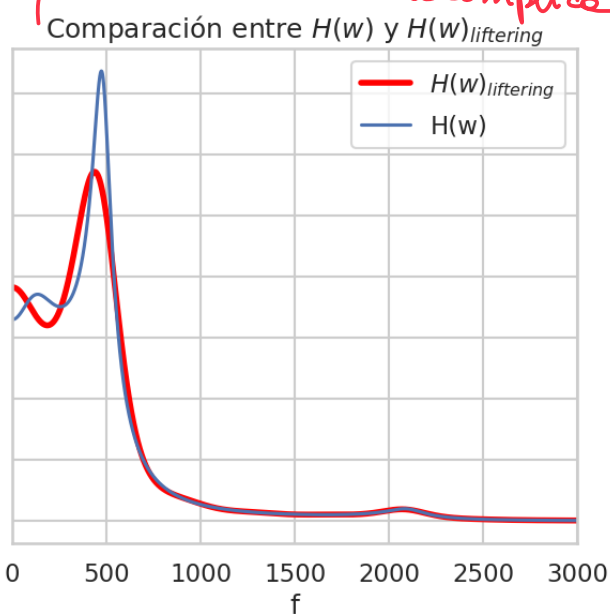
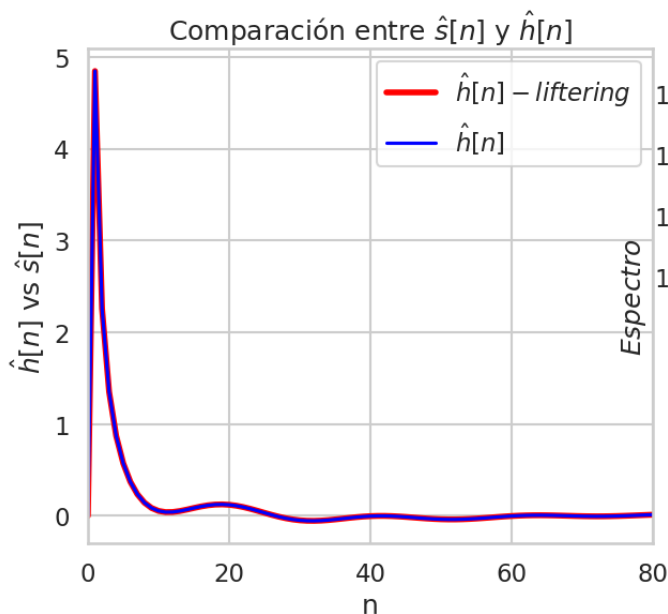
ax[0].plot(h_liftering.real, color='red', lw=4, label=r'$\hat{h}[n]_{\text{liftering}}$')
ax[0].plot(h_cepstrum.real, color='blue', label=r'$\hat{h}[n]$')
ax[0].set_xlim(0, 80)
ax[0].set_xlabel('n')
ax[0].set_ylabel(r'$\hat{h}[n]$ vs $\hat{s}[n]$')
ax[0].set_title(r'Comparación entre $\hat{s}[n]$ y $\hat{h}[n]$')
ax[0].legend()

ax[1].plot(f_cepstrum, np.abs(H_lifterin), color='red', lw=4, label=r'$H(w)_{\text{liftering}}$')
ax[1].plot(f, np.abs(H_syn), label=r'$H(w)$')
ax[1].set_xlim(0, 3000)
ax[1].set_xlabel('f')
```



```
ax[1].set_ylabel(r'$Especro$')
ax[1].set_title(r'$Comparación entre $H(w)$ y $H(w)_{liftering}$')
ax[1].legend()
```

Out[26]: <matplotlib.legend.Legend at 0x7c26ae969790>



cuando están tan cerca F_1 y F_2 se complica

NOTA: Los archivos a entregar son el notebook en python con todo los ítems anteriores y los wavs correspondientes a la vocal sintética del ejercicio 1, el wav de la señal propia grabada para hacer el ejercicio 3, y la señal wav sintetizada a partir de ella correspondientes al punto 3. Los archivos se entregan en el campus, en forma separada, o zipeados (no usar rar). 1

Extra-repito para "i"

Muy bueno !!

In [27]: `#!wget -O fomema_i.wav https://raw.githubusercontent.com/Alex-f98/procHabra20252c/main/brian_i_audio.wav #autocontenido jeje`

```
In [28]: """
# Leer y normalizar
x, Fs = sf.read("fomema_i.wav")
x = x / np.max(np.abs(x))
x_seg_i = x[60_000:124_000][:]

fig, ax = plt.subplots(figsize=(20, 5))
ax.plot(x_seg_i)
ax.set_title("Segmento de la vocal [i] grabada")
ax.set_ylabel("x(n)")
ax.set_xlabel("n")

# Crear zoom-inset
axins = inset_axes(ax, width="40%", height="40%", loc="upper center") # tamaño y ubicación del zoom
axins.plot(x_seg_i)
for freq in [300, 630]:
    axins.axvline(x=freq, color='blue', linestyle='--', alpha=0.7)

x1, x2 = 100, 1000
axins.set_xlim(x1, x2)
axins.set_ylim(-0.7, 0.7)
mark_inset(ax, axins, loc1=2, loc2=4, fc="none", ec="0.5")

"""
```

Out[28]: `'\n# Leer y normalizar\nx, Fs = sf.read("fomema_i.wav")\nx = x / np.max(np.abs(x))\nx_seg_i = x[60_000:124_000][:]\nfig, ax = plt.subplots(figsize=(20, 5))\nax.plot(x_seg_i)\nax.set_title("Segmento de la vocal [i] grabada")\nax.set_ylabel("x(n)")\nax.set_xlabel("n")\n# Crear zoom-inset\naxins = inset_axes(a x, width="40%", height="40%", loc="upper center") # tamaño y ubicación del zoom\naxins.plot(x_seg_i)\nfor freq in [300, 630]:\n axins.axvline(x=freq, colo r='blue', linestyle='--', alpha=0.7)\n\nx1, x2 = 100, 1000\naxins.set_xlim(x1, x2)\naxins.set_ylim(-0.7, 0.7)\nmark_inset(ax, axins, loc1=2, loc2=4, fc="n one", ec="0.5")\n\n'`

In [29]: `print(1/(630-300)*Fs)
#Pero librosa ya me habia dicho 133.44 del ejercicio anterior, deberia cambiar, pregunt0
F0 = 130.44
133.63636363636363`

```
In [30]: #Redefino Nfft dado que hay
Nfft = 4*88192
f = np.linspace(0, Fs, Nfft, endpoint=False)

#Por algun motivo me come toda la RAM!!!!
"""
SXi = np.abs(ft.fft(x_seg_i[:10_000], Nfft))

plt.figure(figsize=(10,4))
plt.plot(f, SXi)

#1er ejercicio
#f_x = [200, 480, 2100]
#o_x = [60 , 100 , 120 ]

f_xi = [100, 480, 2100]
o_xi = [140 , 50 , 100 ]

for freq in f_xi:
    plt.axvline(x=freq, color='red', linestyle='--', alpha=0.7, label=f'Formante en w={freq:.3f}')

plt.xlim((0, 3000))
plt.title("Especro de la vocal [0] real")
plt.xlabel("f")
plt.ylabel("Magnitud ")
plt.grid(True, which='both')
plt.show()
"""
```

?? → después lo mío

